
(IoT 스마트카 시스템) 개발보고서

이름 : 변가은, 이나연, 조혜인

1

개발 목표와 특징

□ 각 팀에서 개발하고자 하는 제안시스템의 특징을 반영한 개발목표를 기술

본 프로젝트는 ABot과 아두이노를 활용하여 라인트래킹, 장애물 감지, 자동주차, 자동소환, 자동 전조등 기능을 통합한 자율주행 보조 시스템을 구현하는 것을 목표로 한다. 하얀색 라인 도로를 기반으로 주행하면서 센서 데이터를 실시간으로 처리해 차량이 상황에 맞게 반응하도록 설계하였다.

특히 초음파 센서, 조도 센서, 블루투스 통신을 결합해 실제 ADAS 기능을 축소한 형태로 구현했으며, 주행 중에는 라인트래킹 센서로 경로를 유지하고, 전방 장애물이 가까워지면 즉시 정지와 경고음을 출력한다.

사용자는 블루투스를 통해 자동주차나 자동소환 명령을 내릴 수 있으며, 차량은 초음파 센서를 이용해 거리 정보를 확인하면서 목표 위치로 이동한다.

또한 주변 밝기가 어두워지면 조도 센서가 이를 감지해 LED 전조등을 자동으로 점등하도록 구현하여 실제 차량과 유사한 환경 반응성을 확보했다.

종합적으로 본 프로젝트는 다양한 센서를 활용해 차량 경로 인식, 장애물 회피, 원격 제어, 환경 감지 등 자율주행 핵심 요소를 직접 실습하고 체험할 수 있다는 점에서 교육적 의미가 있다.

2

개발 필요성

□ 왜 본 시스템 개발이 필요한지 기술적 환경 및 수요 배경을 중심으로 기술

현대 자동차 산업은 완전 자율주행을 목표로 발전하고 있으며, 그 핵심에는 주행 중 위험을 감지하고 스스로 대응하는 능동형 안전 시스템이 자리하고 있다.

본 프로젝트에서는 이러한 기술을 모형 수준에서 구현하기 위해 라인 트레이싱 기반 차선 유지 보조(LKA)와 전방 장애물 감지 시 자동으로 정지하는 AEB(자동 긴급 제동)기능을 적용하였다.

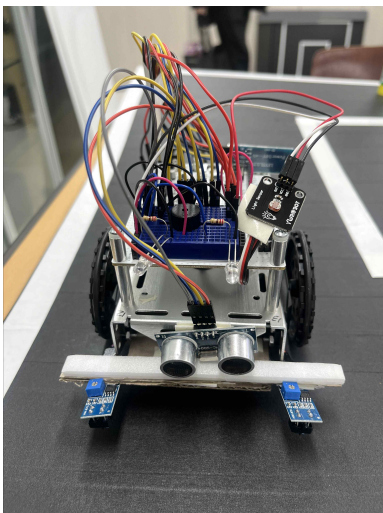
이를 통해 차량이 센서를 통해 주변을 인지하고, 위험 상황을 판단하며, 모터 출력을 직접 제어하는 자율주행의 기본 프로세스를 확인할 수 있었다.

또한, 블루투스 기반 원격 제어 인터페이스를 구축하여 사용자가 스마트폰으로 자동 소환(Summon)자동 주차(Parking)명령을 내릴 수 있도록 설계하였다. 이는 테슬라 Smart Summon이나 현대 RSPA처럼 사용자의 명령과 차량의 자율주행 로직이 결합된 실제 커넥티드 카 기술을 축소 구현한 형태로, 최근 모빌리티 산업의 흐름을 반영한 점에서 프로젝트적 의미가 있다.

3

H/W 구현

기능	장치	핀
블루투스	HC-06 RX → Arduino TX	D5
	HC-06 TX → Arduino RX	D6
앞 초음파	TRIG	D8
	ECHO	D9
뒤 초음파	TRIG	D10
	ECHO	D11
라인트래킹	LEFT	D2
	CENTER	D3
	RIGHT	D4
모터	RIGHT	D12
	LEFT	D13
전조등	LED Left	A0
	LED Right	A1
조도센서	AO(out)	A2
부저	BUZZ	A3



4

S/W 구현

☐ S/W를 어떻게 구현했는지를 설명

☐ 라인트래킹 센서(LEFT_S, RIGHT_S)

흰색/검은색을 디지털 신호로 읽어 차량이 따라가야 할 경로를 실시간으로 판단한다.

☐ 초음파 센서(전면·후면)

- 전면 센서: 장애물 감지 → 일정 거리(8cm 이하)에서는 즉시 급정거

- 후면 센서: 자동주차 시 벽과의 거리를 정확하게 측정하여 후진 깊이를 제어

□ 조도 센서(LDR)

주변 밝기를 읽어 어두워지면 LED 전조등이 자동으로 켜지도록 구현하였다.

□ 라인트래킹

두 센서의 신호 조합에 따라 직진/좌/우 보정 조향/강한 급회전을 수행하는 상태 기반 방식으로 구성하였다.

□ 장애물 감지 및 자동 긴급 정지

전방 초음파 센서 값이 설정값 이하가 될 경우 멈춤+부저음이 출력되도록 구현하였다.

□ 블루투스 기반 사용자 인터페이스

- S: 자동소환

라인트래킹 기반으로 주행하며 사용자 쪽으로 차량이 이동
전방 초음파로 장애물 감지 → 10cm 이내 접근 시 자동 정지
중간에 'X' 명령이 들어오면 즉시 소환 과정 종료

- D: 라인트래킹 주행

라인트래킹 기반으로 지정 경로 주행
앞쪽에 장애물이 10cm 이내 접근하면 자동 정지

- P: 자동주차

후진-왼쪽 커브 진입 -> 앞뒤로 움직임을 총 3회 반복하여 차량을 평행하게 정렬
-> 주차 완료(부저, LED)

- X: 비상 정지

5

구현 소감

□ 개발 과정에 있었던 일과 느낌을 소개(개인별)

□ 변가은

이번 프로젝트를 진행하면서 센서 값이 제대로 읽히지 않아 원인을 찾는 데 어려움이 있었지만, 문제를 해결할 때마다 이해도가 크게 높아졌다. 특히 자동주차 알고리즘을 구현하며 실제 차량 기술이 생각보다 복잡하다는 점을 체감했다. 완성된 로봇이 계획대로 움직이는 모습을 보며 큰 성취감을 얻었다.

□ 이나연

블루투스 통신과 주행 모드를 연동하는 과정에서 여러 번 오작동이 발생해 디버깅에 많은 시간을 썼다. 하지만 그 과정 덕분에 코드 구조를 더 체계적으로 정리할 수 있었다. 자율주행의 핵심 원리를 직접 구현해보면서 이론보다 실습이 훨씬 많은 배움을 준다는 것을 느꼈다.

□ 조혜인

라인트래킹과 장애물 감지를 동시에 처리하는 과정이 쉽지 않았지만, 협력하면서 단계별로 기능을 완성해 나가는 것이 재미있었다. 특히 조도센서를 이용한 자동 전조등 기능을 추가하면서 실제 자동차 기능을 축소 구현한다는 점이 흥미로웠다. 프로젝트를 통해 팀워크의 중요성과 구현 능력을 함께 키울 수 있었다.

6

향후 보완 사항

□ 향후 보완이 필요하다고 생각되는 사항 기술

현재 시스템은 단순한 밝기 기반 라인트래킹 센서를 사용하고 있어, 선이 끊기거나 그림자·조명 변화가 있을 경우 경로를 놓칠 위험이 있다. 이를 개선하기 위해서는 카메라 기반 영상처리(OpenCV 등)를 도입하여 보다 안정적인 차선 인식 알고리즘을 적용할 필요가 있다.

후면 초음파 센서 또한 하나만으로는 실제 차체의 기울기나 양옆 간격을 정확히 판단하기 어렵다. 추가 센서를 도입하여 주차 완성도를 높일 필요가 있다.

7

참고 문헌

□ 개발 과정에 참고한 도서, 사이트 등을 기술

자동주차 참고

<https://www.youtube.com/watchv=5yYpB80yncc>

<https://juahnpop.tistory.com/68>

자동소환 참고

http://www.jkelec.co.kr/img/arduino/line_tracer_manual_2.html

라인 트레이서 참고

<https://myoungjinkim.github.io/arduino/arduino-line-follower-2-dsensors/>

자동 전조등 참고

<https://kocoafab.cc/tutorial/view/754>

초음파센서 거리측정 참고

<https://www.youtube.com/watch?v=D01Apli9qq4>

☐ 스케치 코드를 첨부하세요. 가져온 소스와 AI가 코딩한 소스는 표시하세요.

☐ 블루투스 코드 부분은 코드작성 후 모르는 부분 AI가 도움

```
#include <Servo.h>
```

```
#include <SoftwareSerial.h>
```

```
SoftwareSerial BT(5, 6);
```

```
Servo servoLeft;
```

```
Servo servoRight;
```

```
#define LEFT_S 2
```

```
#define RIGHT_S 4
```

```
#define TRIG_F 8
```

```
#define ECHO_F 9
```

```
#define TRIG_B 10
```

```
#define ECHO_B 11
```

```
#define LED_L A0
```

```
#define LED_R A1
```

```
#define LDR A2
```

```
#define BUZZER A3
```

```
const int SPEED_FWD = 60;
```

```
const int SPEED_TURN_FAST = 80;
```

```
const int SPEED_TURN_ADJUST = 50;
```

```
const int OBSTACLE_DIST = 8;
```

```
const int PARKING_DIST = 8;
```

```
const int BACK_SPEED_L = -55;
```

```
const int BACK_SPEED_R = -35;
```

```
int traceMode = 0;
```

```
void stopMove() {
```

```
    servoLeft.writeMicroseconds(1500);
```

```
    servoRight.writeMicroseconds(1500);
```

```
}
```

```
long getDistance() {
```

```
    digitalWrite(TRIG_F, LOW); delayMicroseconds(2);
```

```
    digitalWrite(TRIG_F, HIGH); delayMicroseconds(10);
```

```
    digitalWrite(TRIG_F, LOW);
```

```
    long duration = pulseIn(ECHO_F, HIGH);
```

```
    return duration * 0.034 / 2;
```

```
}
```

```

long getBackDistance() {
    digitalWrite(TRIG_B, LOW); delayMicroseconds(2);
    digitalWrite(TRIG_B, HIGH); delayMicroseconds(10);
    digitalWrite(TRIG_B, LOW);
    long duration = pulseIn(ECHO_B, HIGH);
    return duration * 0.034 / 2;
}

void maneuver(int left, int right, int ms) {
    servoLeft.writeMicroseconds(1500 + left);
    servoRight.writeMicroseconds(1500 - right);
    if (ms > 0) delay(ms);
}

void runSmartTracing() {
    int L = digitalRead(LEFT_S);
    int R = digitalRead(RIGHT_S);
    long distance = getDistance();

    if (distance > 0 && distance <= OBSTACLE_DIST) {
        stopMove();
        tone(BUZZER, 2000); delay(100); noTone(BUZZER);
        return;
    }

    if (traceMode == 0) {
        if (L == 0) traceMode = 1;
        else if (R == 0) traceMode = 2;
        else maneuver(SPEED_FWD, SPEED_FWD, 20);
    }
    else if (traceMode == 1) {
        if (R == 0) {
            traceMode = 2;
            maneuver(0, 0, 100);
            maneuver(SPEED_TURN_FAST, -80, 400);
        }
        else if (L == 0) maneuver(SPEED_FWD, SPEED_FWD, 20);
        else maneuver(0, SPEED_TURN_ADJUST, 20);
    }
    else if (traceMode == 2) {
        if (L == 0) {
            traceMode = 1;
            maneuver(0, 0, 100);
            maneuver(-80, SPEED_TURN_FAST, 400);
        }
        else if (R == 0) maneuver(SPEED_FWD, SPEED_FWD, 20);
        else maneuver(SPEED_TURN_ADJUST, 0, 20);
    }
}

void autoHeadlight() {

```

```

int lightVal = analogRead(LDR);
if (lightVal < 600) {
    analogWrite(LED_L, 180);
    analogWrite(LED_R, 180);
}
else {
    analogWrite(LED_L, 0);
    analogWrite(LED_R, 0);
}
}

void setup() {
    Serial.begin(9600);
    BT.begin(9600);

    pinMode(LEFT_S, INPUT);
    pinMode(RIGHT_S, INPUT);
    pinMode(TRIG_F, OUTPUT);
    pinMode(ECHO_F, INPUT);
    pinMode(TRIG_B, OUTPUT);
    pinMode(ECHO_B, INPUT);
    pinMode(LED_L, OUTPUT);
    pinMode(LED_R, OUTPUT);
    pinMode(BUZZER, OUTPUT);

    servoLeft.attach(13);
    servoRight.attach(12);

    stopMove();

    tone(BUZZER, 2000, 100); delay(150);
    tone(BUZZER, 3000, 100);

    Serial.println("ABot Ready!");
}

void loop() {

    autoHeadlight();

    if (BT.available()) {

        char cmd = BT.read();
        if (cmd == '\n' || cmd == '\r') return;

        if (cmd == 'X') {
            stopMove();
            tone(BUZZER, 1000, 200);
        }

        else if (cmd == 'S') {

```

```

    traceMode = 0;
    tone(BUZZER, 3000, 100);

    while (true) {
        autoHeadlight();

        long dist = getDistance();
        if (dist > 0 && dist < 10) {
            stopMove();
            for (int i = 0; i < 3; i++) { tone(BUZZER, 2000, 100); delay(150); }
            break;
        }

        runSmartTracing();
        if (BT.available() && BT.read() == 'X') break;
    }
}

else if (cmd == 'D') {
    traceMode = 0;
    tone(BUZZER, 3000, 100);

    while (true) {
        autoHeadlight();
        runSmartTracing();
        if (BT.available() && BT.read() == 'X') break;
    }
}

else if (cmd == 'P') {

    Serial.println("Auto Parking START!");
    tone(BUZZER, 3000, 100);

    Serial.println("Step 1: Curve Back Left...");

    unsigned long startTime = millis();
    while (true) {
        long b_dist = getBackDistance();

        if ((b_dist > 0 && b_dist < PARKING_DIST)) {
            stopMove(); delay(500); break;
        }
        if (millis() - startTime > 2000) {
            stopMove(); break;
        }
    }

    maneuver(-30, -80, 0);
    delay(10);

    if (BT.available() && BT.read() == 'X') return;
}

```



```

}

Serial.println("Step 1.5: Straight Backup...");
while (true) {
    long b_dist = getBackDistance();
    if ((b_dist > 0 && b_dist <= PARKING_DIST) || b_dist == 0) {
        stopMove(); break;
    }

    maneuver(BACK_SPEED_L, BACK_SPEED_R, 0);
    delay(20);

    if (BT.available() && BT.read() == 'X') return;
}
delay(500);

Serial.println("Step 2: Wari-gari (3 times)");

for (int i = 1; i <= 3; i++) {

    maneuver(40, 40, 400);
    stopMove(); delay(300);

    unsigned long backupStart = millis();
    while (true) {
        long b_dist = getBackDistance();

        if ((b_dist > 0 && b_dist <= PARKING_DIST) || b_dist == 0) {
            stopMove(); break;
        }
        if (millis() - backupStart > 2000) {
            stopMove(); break;
        }

        maneuver(BACK_SPEED_L, BACK_SPEED_R, 0);
        delay(20);

        if (BT.available() && BT.read() == 'X') return;
    }

    delay(300);
}

Serial.println("Parking Complete!");
for (int i = 0; i < 3; i++) {
    analogWrite(LED_L, 255);
    analogWrite(LED_R, 255);
    tone(BUZZER, 2000); delay(200);
    analogWrite(LED_L, 0);
    analogWrite(LED_R, 0);
    noTone(BUZZER); delay(200);
}

```

}

}

}

}